



SEPTEMBER 17, 2026

CAPSTONE PROJECT PORTFOLIO

CMSC 447 FALL 2026

MICHAEL J MILLER

Table of Contents

CMSC 447 CAPSTONE PROJECT PORTFOLIO	4
1. COURSE PROJECT OVERVIEW	4
DEVELOPMENT STRUCTURE	4
SPRINT 1 — ARCHITECTURE AND FOUNDATIONS	4
SPRINT 2 — SUBSTANTIAL IMPLEMENTATION AND INTEGRATION	5
SPRINT 3 — SYSTEM INTEGRATION AND DELIVERY	5
2. WHAT WILL BE EVALUATED	6
3. COMMON MULTI-TEAM SCRUM EXPECTATIONS	7
CROSS-TEAM CONTRACTS	7
TEAMS SHOULD MINIMIZE BLOCKING DEPENDENCIES	7
SHARED CHANGES REQUIRE COORDINATION	7
INTEGRATION IS AN ONGOING ACTIVITY	8
4. COMMON ENGINEERING REQUIREMENTS	8
REQUIRED TECHNOLOGY	8
GIT AND GITHUB WORKFLOW	8
CODE REVIEW	9
CONTINUOUS INTEGRATION	9
5. PROJECT OPTIONS	11
PROJECT 1 — NESTED-CLUE CROSSWORD GENERATOR & SOLVER	11
OVERVIEW	11
CORE SYSTEM REQUIREMENTS	11
MAJOR TECHNICAL CHALLENGES	12
TEAM A — WORD & CLUE ENGINE	12
TEAM B — PUZZLE CONSTRUCTION & SOLVING ENGINE	12
TEAM C — WEB APPLICATION & SOLVING EXPERIENCE	13
CROSS-TEAM INTERFACES	13
SPRINT EXPECTATIONS	14
POSSIBLE EXTENSIONS	14

PROJECT 2 — DESTINATION DISPATCH SIMULATION & OPTIMIZATION PLATFORM 15

OVERVIEW	15
INTENDED PRODUCT.....	15
DESTINATION DISPATCH BEHAVIOR.....	16
TEAM A — SIMULATION ENGINE & TRAFFIC GENERATOR	16
TEAM B — DISPATCH & OPTIMIZATION	17
TEAM C — ANALYTICS & EXPERIMENT PLATFORM.....	18
OPTIONAL GROUP AWARENESS	19
PRE-REGISTRATION	19
PUBLIC API INTEGRATION	19
SUGGESTED DATA MODEL	19
SPRINT EXPECTATIONS	21
OPEN DESIGN QUESTIONS / EXTENSIONS.....	21

PROJECT 3 — COURSE PREREQUISITE & ACADEMIC PLANNING PLATFORM.....23

OVERVIEW	23
CORE SYSTEM REQUIREMENTS.....	23
TEAM A — CATALOG & PREREQUISITE ENGINE.....	24
TEAM B — GRAPH ANALYSIS & PLANNING ENGINE	24
TEAM C — STUDENT PLANNING & VISUALIZATION	25
CROSS-TEAM INTERFACES	25
SPRINT EXPECTATIONS	26
POSSIBLE EXTENSIONS.....	26

PROJECT 4 — CROSSCHECK: MULTI-LLM EVALUATION PLATFORM.....27

OVERVIEW	27
INTENDED ARCHITECTURE.....	27
LLM PROVIDER ABSTRACTION	28
TEAM A — EXPERIMENT & MODEL INFRASTRUCTURE.....	28
TEAM B — EVALUATION ENGINE	29
TEAM C — DATASET, ANALYTICS & WEB PLATFORM.....	29
COST AND INFRASTRUCTURE CONSIDERATIONS.....	30
CROSS-TEAM INTERFACES	30
SPRINT EXPECTATIONS	31
POSSIBLE EXTENSIONS.....	31

PROJECT 5 — TURN-BASED SHARED-STATE WORDLE.....33

OVERVIEW	33
CORE REQUIREMENTS	33
TEAM A — GAME MANAGEMENT & BACKEND	33
TEAM B — WORDLE FEEDBACK & CONSTRAINT ENGINE	34
AUTHORITATIVE HISTORY VS. DERIVED STATE	35
TEAM C — WEB GAMEPLAY & PLAYER EXPERIENCE	35
PUBLIC API INTEGRATION	35
CROSS-TEAM INTERFACES	36
SPRINT EXPECTATIONS	36
POSSIBLE EXTENSIONS.....	37
<u>6. FINAL PROJECT SELECTION CONSIDERATIONS.....</u>	<u>38</u>

CMSC 447 Capstone Project Portfolio

1. Course Project Overview

This course project is a substantial software engineering project completed by teams of 3–4 students over approximately nine weeks.

Each project is designed as a **single integrated software system built by 2–3 Scrum teams**. Each Scrum team owns a major subsystem, but the teams must work together to produce one coherent final system.

The purpose is not simply to divide a large application into independent pieces. The project is intended to give you experience with the engineering challenges that arise when multiple teams develop software concurrently:

- defining subsystem boundaries;
- designing APIs and service interfaces;
- establishing shared data models and contracts;
- developing against mocks and stubs so teams can work in parallel;
- coordinating changes across teams;
- integrating independently developed components;
- writing unit, integration, and system tests;
- using Git and GitHub effectively;
- conducting code reviews;
- using continuous integration;
- managing dependencies and shared interfaces;
- diagnosing failures that cross subsystem boundaries; and
- delivering a working system rather than a collection of independent components.

Development Structure

The project consists of **three three-week Scrum sprints**.

The Scrum Master role rotates between team members so that multiple students gain experience facilitating the development process.

Sprint 1 — Architecture and Foundations

Sprint 1 should establish the architecture and allow teams to begin substantial implementation without waiting for other teams.

Teams should:

- define subsystem responsibilities and boundaries;
- identify dependencies between teams;

- agree on API and data contracts;
- create mocks, stubs, fixtures, or test doubles for cross-team dependencies;
- perform technical investigations and spikes where necessary;
- establish the repository, development environment, CI pipeline, and testing framework;
- implement foundational functionality; and
- produce independently testable subsystem prototypes.

The end of Sprint 1 should **not** be a collection of unrelated mockups. Each team should have working technical foundations that demonstrate that its subsystem can be implemented.

Sprint 2 — Substantial Implementation and Integration

Sprint 2 focuses on building the major functionality of the system.

Teams should:

- implement the core behavior of their subsystems;
- replace important mocks with real implementations where appropriate;
- exercise the agreed interfaces;
- begin cross-team integration;
- identify and resolve interface mismatches;
- add integration tests; and
- demonstrate an end-to-end path through the system.

Integration begins during Sprint 2. Teams should not wait until Sprint 3 to discover whether their subsystems work together.

Sprint 3 — System Integration and Delivery

Sprint 3 focuses on turning the integrated system into a reliable final product.

Teams should:

- complete system integration;
 - test realistic end-to-end workflows;
 - address edge cases and failure conditions;
 - improve reliability and usability;
 - test external API failures and other dependencies;
 - improve performance where appropriate;
 - complete documentation;
 - clean up technical debt that affects the final product; and
 - prepare the final demonstration and delivery.
-

2. What Will Be Evaluated

Projects will be evaluated on both the **quality of the resulting software** and the **quality of the engineering process used to produce it**.

The final system should demonstrate:

- meaningful functionality;
- appropriate architecture;
- clear subsystem boundaries;
- usable web interfaces;
- sound relational database design;
- appropriate use of external APIs;
- automated testing;
- integration and system testing;
- reliability and error handling;
- appropriate performance;
- maintainable code; and
- useful documentation.

Because each project is developed by multiple Scrum teams, successful development requires more than having each team's subsystem work independently. Teams must demonstrate that they can build a coherent system across team boundaries.

Evidence used in evaluation may include:

- functionality and completeness;
- architecture and subsystem decomposition;
- API and interface contracts;
- database design;
- automated tests;
- integration/system tests;
- Git and GitHub usage;
- GitHub Issues or equivalent task tracking;
- pull requests;
- code reviews;
- continuous integration;
- cross-team coordination;
- incremental integration;
- documentation;
- handling of failures and edge cases; and
- repository and development history.

The development process is part of the project deliverable. The final repository should reflect the actual development process throughout the three sprints rather than being reconstructed immediately before the final submission.

3. Common Multi-Team Scrum Expectations

Each project will be implemented by 2–3 Scrum teams.

The teams should establish their subsystem boundaries early and maintain those boundaries throughout development.

Cross-Team Contracts

Teams must explicitly identify the interfaces through which their subsystems communicate.

Depending on the project, these may include:

- REST APIs;
- Python service interfaces;
- shared data structures;
- database contracts;
- event/message formats;
- request/response schemas;
- authentication or session contracts; or
- other documented integration points.

Teams should agree on these contracts early enough that each team can develop against mocks or stubs.

Teams Should Minimize Blocking Dependencies

A team should not have to wait for another team to finish an entire subsystem before it can make progress.

For example, if Team B depends on an API owned by Team A, Team A and Team B should agree on the API contract and Team B should be able to use a mock implementation while Team A develops the real implementation.

Shared Changes Require Coordination

Changes to a shared API, database schema, interface, or subsystem boundary can affect multiple teams.

Such changes should be discussed before implementation and should receive appropriate cross-team review.

Integration Is an Ongoing Activity

Teams should integrate incrementally.

A successful project should show evidence of:

Sprint 1: contracts and foundations →

Sprint 2: subsystem integration →

Sprint 3: full system integration and hardening.

4. Common Engineering Requirements

Unless a project description explicitly says otherwise, all projects must use the following technologies and practices.

Required Technology

- **Python backend**
- **Web frontend** using JavaScript or an appropriate JavaScript framework
- **Relational database**, such as PostgreSQL
- **Git and GitHub**
- **Continuous integration**
- **Automated testing**
- **Pull requests and code reviews**
- At least one **public/external API**, where specified by the project

Git and GitHub Workflow

The expected development workflow is:

GitHub Issue → Feature Branch → Implementation → Pull Request → CI → Code Review → Revision → Merge

Repository rules

1. `main` must remain stable. Students should not directly push substantive development work to `main`.
2. Substantive work should be tracked using GitHub Issues. An issue may represent a feature, bug, technical task, investigation, integration task, or other meaningful unit of work.

3. Students should use feature branches for substantive changes.
4. Changes to `main` should go through pull requests.
5. Every pull request must receive meaningful review from at least one other student before it is merged.
6. A student may not be the sole reviewer of their own work.
7. Required CI checks must pass before merging, except where an exception is documented and approved.
8. Pull requests should explain what changed, why it changed, and how the change was tested.
9. Pull requests should reference the relevant GitHub Issue when appropriate.
10. Commits should be meaningful and should avoid unrelated collections of changes.
11. Secrets, API keys, passwords, and other credentials must not be committed to the repository.
12. Virtual environments, caches, IDE files, build artifacts, and other inappropriate generated files should be excluded using `.gitignore`.
13. The repository must contain useful setup and development documentation, including instructions for installation, configuration, database setup, running the application, and running tests.
14. Changes to shared interfaces, APIs, database schemas, or subsystem boundaries require coordination with affected teams.
15. Teams should not bypass the development workflow by manually copying code or making undocumented changes outside the repository.
16. The repository history should reflect the actual development process throughout the project.

Code Review

Every pull request must receive meaningful review from at least one other student.

Reviewers should consider:

- correctness;
- testing;
- maintainability;
- API compatibility;
- error handling;
- adherence to project conventions; and
- potential effects on other subsystems.

Pull requests that modify a shared interface, API contract, database schema, or subsystem boundary should be reviewed by representatives of affected teams when appropriate.

Continuous Integration

The project should use CI to provide automated evidence that changes continue to work.

A typical CI pipeline should include:

1. install dependencies;
 2. run formatting/lint checks;
 3. run unit tests;
 4. run integration tests where appropriate; and
 5. build or otherwise verify the application.
-

5. Project Options

Project 1 — Nested-Clue Crossword Generator & Solver

Overview

Design and implement a crossword puzzle system in which crossword answers can contain **nested clue words**.

In this variant, an answer has a normal clue, but portions of the answer may themselves form valid dictionary words and can have associated sub-clues.

For example, an answer such as:

SCREWDRIVER

may contain valid contiguous words such as:

- SCREW
- CREW
- DRIVE

The system can use these nested words as additional hints. A player might therefore receive a primary clue for the complete answer and progressively receive clues associated with meaningful substrings.

The project combines natural-language data, string-search algorithms, constraint-based puzzle construction, a relational database, and an interactive web application.

Core System Requirements

The system should support:

- a dictionary of candidate words;
- word lookup and validation;
- discovery of valid nested words within larger words;
- primary and nested clues;
- crossword grid construction;
- crossword word-placement constraints;
- puzzle generation;
- puzzle validation;
- interactive solving;
- progressive hinting;
- persistent solve sessions; and
- integration with a public dictionary/definition API.

Major Technical Challenges

A naive implementation could examine every substring independently against the dictionary. Students should investigate more efficient approaches, such as a trie or related multi-pattern/string-search structure.

The crossword generator must also solve a constraint problem:

- words must fit the grid;
- intersecting letters must agree;
- placements must satisfy puzzle constraints;
- generated puzzles must be validated; and
- the system must avoid producing inconsistent or unusable puzzles.

The hint system introduces another relationship: a nested clue must be associated with the correct location and answer context.

Team A — Word & Clue Engine

Team A owns the dictionary and clue-related subsystem.

Responsibilities include:

- dictionary ingestion;
- word lookup;
- word validation;
- trie or related search structure;
- nested-word discovery;
- filtering insignificant or redundant substrings;
- clue generation/management;
- integration with the public dictionary API; and
- caching/error handling for external API calls.

Possible service interfaces include:

```
find_nested_words(answer)
get_clues(word)
validate_word(word)
```

The exact interface should be established jointly with the other teams.

Team B — Puzzle Construction & Solving Engine

Team B owns the computational crossword subsystem.

Responsibilities include:

- grid representation;
- word placement;
- intersection constraints;
- candidate placement generation;
- puzzle generation;
- backtracking/CSP algorithms;
- puzzle validation;
- answer consistency;
- relationships between answers and nested hints; and
- solver support.

This team should expose a clean interface so that the web application does not need to know how the underlying constraint solver works.

Team C — Web Application & Solving Experience

Team C owns the user-facing application.

Responsibilities include:

- puzzle construction interface;
- crossword grid;
- interactive solving;
- clue display;
- progressive hints;
- partial-fill visualization;
- nested-clue visualization;
- hint tracking;
- solve sessions;
- puzzle history; and
- usability.

Cross-Team Interfaces

Teams should agree on contracts for:

- word and clue data;
- puzzle representation;
- grid coordinates;
- answer identifiers;
- nested-word locations;
- hint state;
- puzzle validation results; and
- solve-session state.

The UI should be able to operate against mock puzzle and clue services during Sprint 1.

Sprint Expectations

Sprint 1

- Establish word/dictionary representation.
- Prototype nested-word discovery.
- Establish grid and placement representation.
- Prototype puzzle-generation constraints.
- Build a frontend crossword prototype.
- Agree on word, clue, puzzle, and API contracts.

Sprint 2

- Implement the complete nested-word/clue service.
- Implement meaningful puzzle generation and validation.
- Implement solving and hint behavior.
- Integrate the real services.
- Demonstrate an end-to-end generated puzzle.

Sprint 3

- Handle difficult generation cases.
- Test hint behavior.
- Test invalid and contradictory states.
- Improve persistence and solve history.
- Handle public API failures.
- Improve usability and performance.
- Complete end-to-end testing.

Possible Extensions

- puzzle difficulty estimation;
 - player statistics;
 - timed solving;
 - multiple hint levels;
 - puzzle sharing;
 - generated puzzle themes; or
 - analysis of which nested clues provide the most useful information.
-

Project 2 — Destination Dispatch Simulation & Optimization Platform

Overview

Design and implement a management and simulation platform for a **destination-dispatch elevator system**.

In a conventional elevator system, a rider presses an up/down button in the hallway and then selects a destination after entering the car.

In a destination-dispatch system, the rider declares their destination **before boarding**, and the system determines which specific elevator car should serve them.

The dispatch system therefore knows the destinations of riders in advance and can group compatible riders, route cars more efficiently, and make decisions based on the current state of the building.

The project does not require access to a real elevator system. Instead, students will construct a **simulated building** capable of generating realistic passenger traffic and then use that simulation to evaluate different dispatch strategies and parameter settings.

Intended Product

The system is a management platform for building owners or managers.

A manager should be able to:

- configure a simulated building;
- generate simulated passenger traffic;
- run dispatch simulations;
- view ride and system metrics;
- change dispatch parameters;
- compare system behavior under different settings;
- inspect historical results; and
- correlate system behavior with an external data source.

The required deliverable includes:

- a dispatch simulation engine;
- a synthetic traffic generator;
- a relational database containing persisted ride history;
- a web dashboard;
- configurable dispatch parameters; and

- integration with a real public API.

Destination Dispatch Behavior

A simulated rider enters a destination at a lobby or other floor.

The dispatch algorithm determines which car should serve the request based on factors such as:

- current car positions;
- existing passengers;
- current destinations;
- car capacity;
- expected wait time;
- expected travel time;
- grouping opportunities; and
- configured dispatch parameters.

The system should treat each rider request as an independent transaction. Multiple riders can submit requests without one rider's display or assignment blocking another rider's request.

Inside the car, the simulation can assume that destinations are already known to the system. The simulation should therefore focus on routing and passenger behavior rather than reproducing every physical elevator control.

Team A — Simulation Engine & Traffic Generator

Team A owns the simulated building and event-processing infrastructure.

Responsibilities include:

- configurable building shape;
- number of floors;
- number of elevator cars;
- car capacity;
- elevator state;
- simulated clock;
- passenger state;
- ride requests;
- event processing;
- car movement;
- passenger boarding and exiting;
- synthetic traffic generation; and
- reproducible simulations using random seeds.

Traffic profiles should include several distinct patterns, such as:

- **morning up-peak** — most requests originate in the lobby;
- **lunch bidirectional** traffic;
- **evening down-peak**;
- **random daytime baseline**; and
- **stress/burst traffic**.

Each simulated rider should generate information such as:

- origin floor;
- destination floor;
- request time;
- optional group information; and
- optional pre-registration information.

The simulator should operate on **simulated time**, allowing many building days to be generated and analyzed quickly rather than requiring real-time execution.

Team B — Dispatch & Optimization

Team B owns the dispatch algorithm.

Responsibilities include:

- assigning requests to cars;
- tracking car capacity;
- grouping compatible riders;
- batching requests;
- handling peak traffic;
- idle-car repositioning;
- implementing multiple dispatch strategies; and
- evaluating the trade-offs between different objectives.

The dispatch engine should expose a clean decision interface to the simulation engine.

The project should include configurable parameters that affect dispatch behavior.

Possible parameters include:

Maximum passengers grouped per car

Controls how many distinct requests can be batched into one car trip.

Wait-vs.-travel weighting

Controls the relative emphasis on minimizing time until pickup versus minimizing total door-to-door travel time.

Batching window

Controls how long the algorithm waits for additional compatible requests before dispatching.

Up-peak / down-peak thresholds

Request-rate thresholds can trigger specialized peak-handling behavior.

Idle-car pre-positioning

Controls whether idle cars redistribute toward historically high-demand floors between requests and how aggressively they do so.

Changes to parameters should be attributable to particular periods in the simulation so that results can be compared before and after a configuration change.

Team C — Analytics & Experiment Platform

Team C owns persistence, experiment management, analytics, and the web dashboard.

Responsibilities include:

- experiment configuration;
- simulation execution requests;
- persistence;
- experiment history;
- ride logs;
- metric calculation;
- dashboard visualizations;
- scenario comparison;
- parameter history;
- before/after comparisons; and
- management-oriented presentation of results.

Potential metrics include:

- average wait time;
- maximum wait time;
- percentile wait time;
- average travel time;
- total travel time;
- number of stops;
- elevator utilization;
- throughput;
- total rides; and
- grouping efficiency.

Every ride should be persisted with enough information to support analysis without requiring the dashboard to reconstruct everything from raw simulation events.

Optional Group Awareness

If multiple riders request the same origin/destination within a short simulated window, the system may treat them as a group for dispatch and analytics.

This allows the system to distinguish individual efficiency from group efficiency and provides a concrete opportunity to evaluate the effect of batching.

Pre-Registration

A secondary feature is **pre-registration**, rather than a traditional reservation system.

A rider may indicate something such as:

heading to Floor 12 in approximately 5 minutes

This does not reserve a car or hold a time slot. Instead, it provides the dispatch system with an early, lower-confidence signal that may be used to reposition idle cars.

If the rider does not arrive within an appropriate grace period, the pre-registration expires without holding resources.

The system should record whether a completed ride was preceded by a pre-registration so that the dashboard can examine whether the feature affects wait times under sufficiently heavy demand.

Public API Integration

Because the core simulation does not inherently require an external API, the project must deliberately incorporate an external signal.

Possible integrations include:

- a building occupancy/calendar API to correlate scheduled meetings with elevator demand;
- a weather API to correlate weather conditions with changes in traffic; or
- a public transit API to examine relationships between transit arrivals and building traffic.

The external API should be a genuine system dependency rather than merely a decorative data source.

Suggested Data Model

A starting point for the relational database might include:

buildings

- id
- name
- num_floors
- num_cars
- car_capacity
- created_at

cars

- id
- building_id
- capacity

rides

- id
- building_id
- car_id
- group_id
- origin_floor
- destination_floor
- requested_at
- car_arrived_at
- destination_reached_at
- was_pre_registered
- stops_during_ride

ride_groups

- id
- building_id
- formed_at

pre_registrations

- id
- building_id
- rider_token
- destination_floor
- registered_at
- expected_departure_at
- expired
- matched_ride_id

dispatch_parameter_sets

- id
- building_id
- max_grouped_passengers
- wait_travel_weight
- batching_window_seconds
- up_peak_threshold
- down_peak_threshold
- idle_repositioning_enabled
- effective_from
- effective_to

metrics_daily

```
building_id
date
avg_wait_time
avg_travel_time
p95_wait_time
total_rides
avg_stops_per_trip
dispatch_parameter_set_id
```

This schema is a starting point rather than a prescribed design. In particular, parameter sets with effective time windows allow historical rides to be associated with the exact configuration that was active when they occurred, making tuning decisions auditable.

Sprint Expectations

Sprint 1

- Build the basic event-driven simulator.
- Establish building, car, passenger, and request models.
- Establish the dispatch-decision interface.
- Implement an initial dispatch strategy.
- Prototype experiment configuration and dashboard using mock results.
- Establish shared simulation-state, request, dispatch-decision, and result contracts.

Sprint 2

- Implement realistic traffic profiles and elevator behavior.
- Implement multiple dispatch strategies or meaningful strategy variations.
- Implement persistence and experiment history.
- Implement dashboard visualizations.
- Integrate actual simulation results with the analytics system.
- Demonstrate at least two dispatch strategies through the integrated platform.

Sprint 3

- Compare scenarios and parameter configurations.
- Test reproducibility.
- Exercise high-volume and burst traffic.
- Test dispatch failures and invalid configurations.
- Improve simulation and dashboard performance.
- Complete visualization and end-to-end testing.
- Demonstrate how changing parameters affects measured outcomes.

Open Design Questions / Extensions

Possible extensions include:

- side-by-side comparison of multiple "what-if" parameter sets;
- more sophisticated pre-registration behavior;
- rider abandonment;
- real-time dashboard updates;
- additional dispatch strategies;
- optimization experiments;
- more detailed elevator physics; or
- additional external signals.

Real-time dashboard updates, for example, could use WebSockets, but periodic refresh is sufficient for the core project.

Project 3 — Course Prerequisite & Academic Planning Platform

Overview

Design and implement a web platform that helps students understand course prerequisites and construct valid multi-semester academic plans.

Unlike a simple prerequisite checker, the system must represent **complex prerequisite relationships**, including combinations such as:

(A AND B) OR C

The platform should allow users to explore course dependencies, determine which courses they are currently eligible to take, identify the consequences of taking or delaying a course, and generate possible semester-by-semester plans.

The planning problem is therefore more complicated than simply performing a topological sort. A valid plan may need to account for:

- completed courses;
- prerequisite relationships;
- course offerings;
- maximum course loads;
- degree requirements;
- semester boundaries;
- courses that cannot be taken simultaneously; and
- other planning constraints.

Core System Requirements

The system should support:

- course catalog management;
- complex AND/OR prerequisite structures;
- prerequisite evaluation;
- course-offering information;
- dependency graphs;
- cycle detection;
- reachability analysis;
- eligibility analysis;
- semester planning;
- student course history;
- degree requirements;

- plan generation;
- manual plan modification; and
- visualization.

Team A — Catalog & Prerequisite Engine

Team A owns the course and prerequisite data model.

Responsibilities include:

- course catalog CRUD;
- course identifiers and descriptions;
- prerequisite representation;
- AND/OR prerequisite groups;
- prerequisite evaluation;
- course offerings;
- degree-requirement data;
- public catalog API; and
- validation of catalog data.

The prerequisite representation must be capable of expressing structures such as:

(A AND B) OR C

Possible service interfaces include:

```
is_eligible(course, completed_courses)
evaluate_prerequisites(course, completed_courses)
get_course(course_id)
get_offerings(course_id)
```

The exact representation and API should be agreed upon with Team B.

Team B — Graph Analysis & Planning Engine

Team B owns graph algorithms and plan generation.

Responsibilities include:

- prerequisite dependency graphs;
- graph traversal;
- cycle detection;
- reachability;
- downstream impact analysis;
- eligibility frontier calculation;
- constrained course ordering;
- semester plan generation;

- course-load constraints;
- offering constraints; and
- detection of invalid plans.

The planning engine should be independent of the web interface.

For example, the planning engine should be able to receive a student's completed coursework and planning constraints and return a structured plan without knowing how that plan will be displayed.

Team C — Student Planning & Visualization

Team C owns the user-facing application.

Responsibilities include:

- course catalog browsing;
- prerequisite visualization;
- student profile;
- completed-course selection;
- plan builder;
- generated-plan display;
- manual overrides;
- plan validation;
- dependency/impact visualization;
- semester organization; and
- plan history/versioning.

The interface should make complex prerequisite relationships understandable rather than merely displaying raw graph data.

Cross-Team Interfaces

Teams should agree on:

- course representation;
- prerequisite-expression format;
- prerequisite evaluation results;
- graph representation;
- eligibility results;
- planning constraints;
- generated-plan format; and
- validation/error messages.

The UI should be able to develop against mock course and planning data during Sprint 1.

Sprint Expectations

Sprint 1

- Establish the course/catalog model.
- Establish AND/OR prerequisite representation.
- Implement basic prerequisite evaluation.
- Prototype dependency graph representation.
- Investigate graph-analysis algorithms.
- Build catalog and visualization prototypes.
- Agree on course, prerequisite, graph, and evaluation contracts.

Sprint 2

- Implement complete prerequisite evaluation.
- Implement eligibility and graph analysis.
- Implement initial semester-plan generation.
- Implement student state and completed-course tracking.
- Integrate the catalog, planning engine, and UI.

Sprint 3

- Handle complex AND/OR cases.
- Detect cycles and invalid prerequisite structures.
- Add impact analysis.
- Support plan regeneration.
- Support manual overrides.
- Add plan versioning/history.
- Test invalid plans and unusual course-offering cases.
- Complete end-to-end testing and usability improvements.

Possible Extensions

- degree-program templates;
 - "what happens if I delay this course?" analysis;
 - alternative plans;
 - graduation-date estimation;
 - prerequisite bottleneck detection;
 - course-load recommendations;
 - plan comparison; or
 - visualization of courses that are on the critical path toward a degree requirement.
-

Project 4 — CrossCheck: Multi-LLM Evaluation Platform

Overview

Design and implement a software platform for evaluating the **consistency, agreement, and potential errors of responses produced by multiple large language models**.

The system should allow an instructor, researcher, or other user to construct reproducible evaluation experiments.

A basic experiment might provide the same task to several language models and then ask an evaluator to compare their responses.

However, **agreement is not the same as correctness**. Multiple models can agree on an incorrect answer, and an evaluator model can itself make an incorrect judgment.

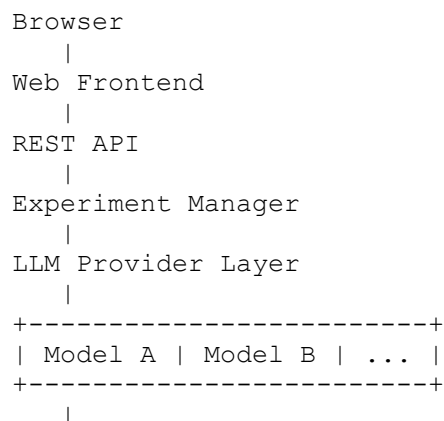
The platform should therefore support multiple forms of evaluation, including:

- model-to-model comparison;
- cross-evaluation;
- consensus analysis;
- comparison with reference answers;
- programmatic verification where appropriate; and
- structured evaluation results.

The project is therefore not simply "have ChatGPT critique Claude." It is a reusable infrastructure for conducting multi-model evaluation experiments.

Intended Architecture

A conceptual architecture is:



```
Evaluation Engine
|
Structured Results
|
PostgreSQL
|
Web Interface / Analytics
```

LLM Provider Abstraction

The system should not hard-code the application around one specific model vendor.

A provider abstraction might look conceptually like:

```
class LLMProvider(ABC):

    @abstractmethod
    async def generate(
        self,
        prompt: str,
        options: GenerationOptions
    ) -> LLMResponse:
        pass
```

Possible implementations include:

- a mock provider;
- a locally hosted model;
- an externally hosted model; or
- other provider implementations.

The project should be designed so that the underlying inference backend can change without rewriting the rest of the application.

The system must communicate with at least one LLM through a well-defined API/interface. The LLM may be locally hosted or externally hosted.

Team A — Experiment & Model Infrastructure

Team A owns experiment execution and model-provider infrastructure.

Responsibilities include:

- experiment definitions;
- task execution;
- provider abstraction;
- model configuration;
- job management;
- response storage;

- reproducibility;
- provider failures;
- timeouts;
- retries where appropriate; and
- execution status.

The system should make it possible to run the same evaluation task repeatedly and identify which models/configurations participated in each experiment.

Team B — Evaluation Engine

Team B owns the evaluation logic.

Responsibilities include:

- evaluator architecture;
- pairwise comparison;
- cross-evaluation;
- consensus analysis;
- reference-answer comparison;
- structured evaluation results;
- evaluator confidence;
- disagreement identification; and
- analysis of evaluator reliability.

A structured evaluator result might contain:

```
overall_agreement
disagreements[]
potential_errors[]
evaluator_confidence
```

The exact schema should be designed jointly with Team C.

The team should explicitly consider the limitations of using one LLM to evaluate another. The evaluation system should distinguish between:

- model agreement;
- evaluator judgment;
- reference-answer agreement; and
- programmatically verified correctness.

Team C — Dataset, Analytics & Web Platform

Team C owns datasets, result presentation, persistence-facing functionality, and the web interface.

Responsibilities include:

- datasets;
- questions/tasks;
- reference answers;
- experiment history;
- result aggregation;
- statistics;
- visualizations;
- experiment comparison;
- response comparison;
- disagreement display; and
- result exploration.

The UI should allow a user to move from an experiment-level summary into the individual tasks and responses that produced the result.

Cost and Infrastructure Considerations

The project should be capable of substantial development at little or no monetary cost where possible.

Possible approaches include:

- mock providers;
- locally hosted models;
- university-provided inference resources;
- limited externally hosted APIs.

During Sprint 1, teams should investigate the inference resources available to them.

The system should be designed so that the choice of inference backend does not fundamentally change the application architecture.

Cross-Team Interfaces

Teams should agree on:

- experiment representation;
- task representation;
- model configuration;
- provider interface;
- LLM response format;
- evaluator input/output format;
- reference-answer representation;
- evaluation-result schema; and

- experiment status.

Mocks and fixtures should allow Teams B and C to work before Team A has completed integration with a real model.

Sprint Expectations

Sprint 1

- Establish experiment/task models.
- Build the provider abstraction.
- Implement a mock provider.
- Investigate available local or external inference resources.
- Define evaluator interfaces.
- Create representative evaluation fixtures.
- Prototype dataset/results UI using mock data.
- Agree on experiment, task, response, and evaluation schemas.

Sprint 2

- Integrate at least one real model.
- Implement meaningful evaluation strategies.
- Implement experiment history and result persistence.
- Implement visualization and comparison.
- Integrate the complete Task → Models → Evaluator → Results pipeline.

Sprint 3

- Support multiple evaluation strategies.
- Compare experiments.
- Analyze disagreements.
- Compare against reference answers.
- Handle provider failures, timeouts, and malformed responses.
- Improve reproducibility.
- Add useful analytics.
- Test end-to-end evaluation workflows.
- Improve performance and usability.

Possible Extensions

- multiple evaluators;
- evaluator agreement analysis;
- human evaluation;
- programmatic answer verification;
- confidence calibration;
- prompt-version tracking;

- model-version tracking;
 - benchmark datasets;
 - cost/latency analysis; or
 - statistical analysis of disagreement patterns.
-

Project 5 — Turn-Based Shared-State Wordle

Overview

Design and implement a two-player, turn-based version of Wordle in which the players share a single accumulated constraint state.

The players alternate submitting guesses against one hidden five-letter target.

Unlike ordinary Wordle, the important system-level challenge is that **all information learned by either player becomes part of one shared global state**.

A player's guess can therefore affect what the other player is allowed to submit later.

Players may submit valid guesses that do not reveal new information. Such "stalling" guesses are legal; the system must not assume that every guess necessarily tightens the constraint set.

The project combines game-state management, server-authoritative state, relational persistence, a nontrivial feedback algorithm, constraint accumulation, and interactive multiplayer UI.

Core Requirements

The system should support:

- game creation;
- player joining;
- alternating turns;
- a hidden target word;
- valid-word checking;
- standard Wordle feedback;
- repeated-letter handling;
- global constraint accumulation;
- candidate validation;
- persistent game state;
- game history;
- shared board state;
- player status; and
- a public word/dictionary API.

Team A — Game Management & Backend

Team A owns the game lifecycle and server-authoritative state.

Responsibilities include:

- game creation;
- player registration/joining;
- player identity;
- turn management;
- game lifecycle;
- target-word selection;
- REST API;
- database persistence;
- server-side validation;
- game history;
- reconnect/resume behavior; and
- synchronization of shared state.

The server, rather than the browser, must be authoritative about:

- whose turn it is;
- whether a guess is legal;
- the current game state;
- the target;
- and the accumulated constraints.

Team B — Wordle Feedback & Constraint Engine

Team B owns the computational core.

Responsibilities include:

- Wordle feedback calculation;
- repeated-letter semantics;
- constraint representation;
- constraint accumulation;
- candidate validation;
- reconstruction of constraints from history;
- consistency checking; and
- extensive edge-case testing.

Conceptual interfaces include:

```
evaluate_guess(target, guess)
    -> feedback
```

```
update_constraints(previous_state, feedback)
    -> new_state
```

```
validate_candidate(word, constraints)
    -> valid / invalid
```

Repeated letters are especially important.

For example, feedback cannot simply be computed by independently labeling each occurrence of a letter. The implementation must correctly account for how many copies of each letter occur in the target and how those copies are allocated among exact-position and present-but-misplaced matches.

Authoritative History vs. Derived State

The historical record of guesses and feedback should be treated as authoritative.

The accumulated constraint state may be cached for efficiency, but the system should be capable of reconstructing it from the game history.

This gives the project an important state-management requirement:

If the derived constraint state is lost or corrupted, the system should be able to rebuild it from the authoritative game record.

Team C — Web Gameplay & Player Experience

Team C owns the interactive game interface.

Responsibilities include:

- shared game board;
- player indicators;
- turn indicator;
- keyboard;
- guess entry;
- feedback visualization;
- global constraint visualization;
- invalid-guess feedback;
- game history;
- game status;
- player status;
- statistics; and
- reconnect/resume experience.

The UI should clearly communicate that the constraint state is shared rather than belonging to one player.

Public API Integration

The project should integrate with a public dictionary/word API for one or more appropriate functions, such as:

- validating candidate words;
- obtaining possible target words; or
- obtaining definitions.

The external API should be isolated behind a service interface so that the game remains testable when the external service is unavailable.

Mocks should be used during development and testing.

Cross-Team Interfaces

Teams should agree on:

- game-state representation;
- turn representation;
- guess request/response format;
- feedback representation;
- constraint representation;
- player status;
- game lifecycle states; and
- error responses.

Team C should be able to build against a mock game API while Teams A and B implement the actual backend logic.

Sprint Expectations

Sprint 1

- Establish the game model and database schema.
- Build basic game/API foundations.
- Implement Wordle feedback with repeated-letter tests.
- Establish the constraint representation.
- Build a shared-board UI using mock data.
- Agree on game-state, turn, feedback, and constraint contracts.

Sprint 2

- Implement complete server-side game management.
- Implement constraint accumulation and candidate validation.
- Complete the gameplay interface.
- Integrate the actual shared game state.
- Demonstrate a complete two-player game.

Sprint 3

- Test difficult repeated-letter cases.
- Test constraint reconstruction.
- Test reconnect and abandonment behavior.
- Test external API failures.
- Improve synchronization and usability.
- Add history/statistics.
- Complete end-to-end testing.

Possible Extensions

Possible extensions include:

- best-of-N matches;
 - alternating first player;
 - leaderboards;
 - reconnect/resume;
 - spectator mode;
 - game replay;
 - candidate-word counts;
 - information-gain analysis; or
 - tournament play.
-

6. Final Project Selection Considerations

The five projects emphasize different kinds of software engineering problems.

When selecting a project, consider the kinds of engineering work you would like to experience:

- **Nested-Clue Crossword** — search algorithms, constraint solving, puzzle generation, and interactive UI.
- **Destination Dispatch** — simulation, optimization, event processing, analytics, and experimentation.
- **Academic Planning** — graph algorithms, complex data modeling, constraint-based planning, and visualization.
- **CrossCheck** — distributed/API-oriented architecture, LLM integration, evaluation methodology, structured results, and analytics.
- **Shared-State Wordle** — state management, multiplayer interaction, algorithmic correctness, persistence, and synchronization.

No project is intended to be implemented by one team in isolation. In every case, the central software engineering challenge includes coordinating multiple teams to build and integrate a single system.

The project should therefore be evaluated not only by whether the final application works, but also by whether the teams demonstrate the engineering practices necessary to build a nontrivial system collaboratively.